Progettazione di un sistema di gestione bibliotecaria

Esercizio di Object-Oriented Design | Livello intermedio | Nessun codice eseguibile richiesto

Scenario

Una rete di biblioteche comunali vuole sviluppare un software per gestire cataloghi, utenti iscritti, prestiti, prenotazioni e notifiche. Attualmente molte operazioni sono manuali e causano prestiti non registrati, prenotazioni duplicate e difficoltà nel verificare la disponibilità dei materiali.

Obiettivo: progettare le classi e le relazioni necessarie per un sistema centralizzato, estensibile e manutenibile. Non devi implementare il funzionamento dei metodi.

1. Gestione delle biblioteche

Il sistema gestisce più biblioteche. Ogni biblioteca possiede nome, indirizzo, orari di apertura e catalogo. Un materiale può essere presente in più biblioteche, ciascuna con diverse copie fisiche. Deve essere possibile individuare dove si trova una copia, verificarne la disponibilità e trasferirla tra biblioteche, mantenendo traccia del trasferimento.

2. Gestione dei materiali

- Libri cartacei: titolo, autore, ISBN e numero di pagine.
- Riviste: titolo, numero dell’edizione e data di pubblicazione.
- DVD: titolo, durata e classificazione per età.
- Libri digitali: titolo, autore, formato e dimensione del file.

Le regole variano: i libri cartacei possono essere prestati per 30 giorni, i DVD per 7 giorni; i libri digitali sono consultabili per un periodo limitato senza restituzione fisica. Alcuni materiali sono consultabili soltanto in sede. In futuro saranno aggiunte nuove tipologie.

3. Gestione degli utenti

Ogni utente ha identificativo univoco, nome, cognome, email e data di registrazione. Sono previste le categorie utente standard, studente universitario, docente e bibliotecario. Un utente standard può avere al massimo 3 prestiti attivi, uno studente 5 e un docente 10. Alcune categorie hanno durate di prestito maggiori. I bibliotecari registrano prestiti e restituzioni e modificano il catalogo. Devono poter essere introdotte nuove categorie senza riscrivere la gestione dei prestiti.

4. Gestione dei prestiti

Prima di autorizzare un prestito il sistema verifica disponibilità del materiale, limite di prestiti dell’utente, eventuali prestiti scaduti bloccanti, prestabilità del materiale e requisiti specifici. Ogni prestito conserva data di inizio, scadenza, eventuale restituzione e stato. Sono possibili proroghe quando consentite; le regole possono variare in base al materiale e alla categoria di utente.

5. Gestione delle prenotazioni

Se tutte le copie sono occupate, gli utenti possono prenotare un materiale. Le richieste sono inizialmente gestite in ordine cronologico. Quando una copia torna disponibile, il primo utente in attesa riceve una notifica e dispone di un periodo limitato per ritirarla. Se il termine scade, il sistema passa all'utente successivo. Le prenotazioni possono essere annullate finché non completate. Lo stesso utente non può avere più prenotazioni attive per il medesimo materiale.

6. Gestione delle notifiche

Il sistema avvisa gli utenti per prestiti in scadenza o scaduti, materiali prenotati disponibili, prenotazioni prossime alla scadenza e proroghe approvate o rifiutate. Sono supportati email, SMS e notifiche interne. Ogni utente può scegliere uno o più canali preferiti. Deve essere possibile aggiungere nuovi canali senza modificare le funzionalità che generano gli avvisi.

7. Gestione delle penalità

I ritardi possono generare penalità in base a tipo di materiale, giorni di ritardo e categoria dell'utente. Alcune categorie possono avere giorni di tolleranza. Le penalità possono essere economiche o prevedere la sospensione temporanea dei prestiti. Le regole devono poter cambiare senza moltiplicare controlli specifici per ogni combinazione di utente e materiale.

Consegna

Progetta l'architettura necessaria a soddisfare i requisiti. Per ogni classe o interfaccia indica:

- Nome e responsabilità.
- Attributi con tipi e visibilità.
- Metodi con parametri, tipi di ritorno e breve descrizione del comportamento.
- Relazioni con gli altri componenti: ereditarietà, composizione, aggregazione o dipendenza.
- Eventuali classi astratte, interfacce e generics, motivando quando sono utili.

Puoi utilizzare pseudocodice, UML o la sintassi del linguaggio che conosci meglio. Non è necessario implementare i metodi.

Vincoli progettuali

- Assegna responsabilità ben definite alle classi; evita classi che fanno tutto.
- Riduci le dipendenze dirette e la duplicazione di logica.
- Proteggi lo stato interno degli oggetti tramite incapsulamento.
- Usa ereditarietà solo per reali relazioni di specializzazione.
- Preferisci soluzioni estensibili, senza modifiche continue al codice esistente.
- Non introdurre astrazioni o generics senza un'esigenza concreta.

Scenari aggiuntivi

A — Nuovi materiali. Si introducono tablet e computer portatili, con restituzione entro 48 ore e deposito cauzionale.

B — Nuove notifiche. Si aggiungono notifiche WhatsApp, mantenendo email, SMS e notifiche interne.

C — Utenti premium. Un abbonamento acquistabile da qualsiasi categoria aumenta il numero massimo di prestiti e consente proroghe aggiuntive.

D — Prenotazioni prioritarie. Alcuni utenti hanno priorità su determinati materiali, senza lasciare indefinitamente in attesa gli altri.

E — Nuove penalità. Le penalità economiche vengono sostituite da punti; superata una soglia, l'utente viene temporaneamente sospeso.

Per ciascuno scenario descrivi quali componenti aggiungeresti o modificheresti e spiega perché.

Elaborato richiesto

- Elenco delle classi e interfacce con le rispettive responsabilità.
- Attributi e firme dei metodi.
- Diagramma UML delle classi con le relazioni principali.
- Motivazione di almeno tre decisioni architetturali e delle alternative scartate.
- Descrizione di come l'architettura affronta i cinque scenari aggiuntivi.

Nota: la valutazione privilegia coerenza, chiarezza e motivazione delle scelte, non il numero di classi o pattern utilizzati.